UNIVERSIDAD AMERICANA
Facultad de Ingeniería y Arquitectura



Servicios Web

Práctica de Seguridad con SpringBoot

Estudiantes:

José Ariel Zeledón Iglesias.	**Cif**: 23010244
Rodrigo Dariel Romero González	**Cif**: 21012438

Fecha: 22/10/2026

Managua, Nicaragua

Preguntas de cierre

Práctica 9: Seguridad con JWT en NicaMarket API · Servicios Web

1. ¿En qué se diferencian HTTP 401 y 403?

El 401 aparece cuando la API no sabe quiénes somos. Por ejemplo, cuando hicimos la consulta de productos sin mandar el token, o cuando probamos con un token que nosotros mismos modificamos, la respuesta fue 401.

El 403 es diferente: la API sí sabe quiénes somos, pero no nos deja hacer esa acción. Cuando entramos con el usuario operador (que tiene rol USER) pudimos ver los productos sin problema, pero al intentar crear un producto o una categoría nos dio 403, porque eso solo lo puede hacer el ADMIN.

Con esto entendimos la diferencia entre autenticación y autorización. La autenticación es comprobar quién es el usuario (el login y el token), y la autorización es revisar si ese usuario tiene permiso para lo que quiere hacer (según su rol).

2. ¿Por qué no basta con leer el contenido de un JWT para confiar en él?

Porque el contenido de un JWT no está oculto, solo está codificado. Cualquier persona puede copiar el token, decodificarlo en una página como jwt.io y ver el usuario, el rol y la fecha de vencimiento. Así como se puede leer, también se puede cambiar, por ejemplo poner que el rol es ADMIN.

Lo que hace que el token sea confiable es la firma. La firma se genera con una clave secreta que solo tiene la API, así que si alguien cambia algo del token, la firma ya no coincide y la API lo rechaza. Eso lo comprobamos en Postman con la prueba del token modificado, que devolvió 401.

3. ¿Qué función cumple JwtDecoder?

JwtDecoder es la parte de Spring Security que revisa el token cada vez que llega una petición. Básicamente es el que decide si el token es válido o no. En el proyecto revisa tres cosas:

- que la firma sea correcta, usando la misma clave con la que se creó el token;
- que el token no esté vencido (en nuestro caso dura 30 minutos);
- que el token lo haya emitido nuestra API (nicamarket-laboratorio).

Si algo de eso falla, la API responde 401. Si todo está bien, se leen los roles del token y

con eso Spring Security decide si el usuario puede hacer la operación o si le responde 403.

4. ¿Qué riesgos ocasionaría publicar la clave utilizada para firmar tokens?

Sería muy grave, porque con esa clave cualquiera podría crear sus propios tokens y la API los aceptaría como si fueran reales. Por ejemplo, alguien podría hacerse un token con rol ADMIN sin saber la contraseña y crear, modificar o borrar productos y categorías. También podría hacerse pasar por otro usuario o ponerle al token un vencimiento muy largo para tener acceso por mucho tiempo.

Además, si la clave se sube a GitHub aunque sea una vez, queda en el historial, y borrar el archivo después no sirve. Habría que cambiar la clave. Por eso en este proyecto la clave no está en el código: se configura como variable de entorno (JWT_SECRET_BASE64) en IntelliJ y no se sube al repositorio.